

Week 3 Report - Hybrid Recommender System

Week 3 Report – Hybrid Recommender System

1. Introduction

This report presents the third stage of the recommender system development, focusing on improving personalization, strengthening data representation, and enabling deployment capabilities.

The system evolves from basic recommendation techniques (Week 2) to a hybrid model that integrates multiple signals for better performance.

The objectives of Week 3 are:

- Enhance data quality through feature engineering
- Improve recommendation performance using hybrid modeling
- Enable real-time recommendation access through API deployment

2. Methodology

2.1 Feature Engineering

Raw interaction data was transformed into meaningful numerical signals to better represent user preferences.

Interaction weights were defined as:

- View → 1 (low intent)
- Cart → 5 (medium intent)
- Purchase → 10 (high intent)

These values capture the intensity of user interest.

The data was then aggregated at the (user_id, product_id) level:

- This reduces noise from repeated interactions
- Produces a stronger signal for recommendation models

Result:

Each user-product pair now has a weighted score representing preference strength.

2.2 Hybrid Recommendation Approach

A hybrid recommendation model was implemented by combining collaborative filtering with interaction scoring.

Process:

1. Construct user-item matrix using interaction scores
2. Compute cosine similarity between users
3. Identify top similar users for each target user

4. Aggregate scores of similar users
5. Rank products based on aggregated scores
6. Return top-N recommendations

Advantages:

- Combines personalization (user similarity)
- Uses interaction intensity (weighted scores)
- Handles sparse data better than standalone models

2.3 API Deployment

A REST API was developed using FastAPI to make the system usable in real-world applications.

Endpoints:

- GET /

Returns system status

- GET /recommend/{user_id}

Returns top-N recommendations for the given user

Features:

- Fast response time
- Lightweight deployment
- Easy integration with frontend or applications

2.4 Evaluation Metrics

Advanced evaluation metrics were introduced to better measure ranking performance:

Recall@K:

Measures how many relevant items are successfully recommended.

MAP@K (Mean Average Precision):

Measures ranking quality by rewarding correct items appearing earlier in recommendations.

These metrics provide better insight than simple precision-based evaluation.

3. Results

Performance Summary (qualitative):

Hybrid Model:

- Produces more personalized recommendations
- Improves ranking of relevant items
- Better handles sparse interaction data

Comparison with Week 2:

- Outperforms popularity model in personalization
- Improves over collaborative filtering by using weighted signals
- Provides more stable and meaningful recommendations

4. Observations

- Interaction weighting significantly improves model performance
- Hybrid approach reduces impact of sparse datasets
- Similar users contribute strong recommendation signals
- Ranking quality improves compared to earlier models
- API integration enables practical usage of the system
- System remains computationally efficient for medium-scale datasets

5. Sample Recommendations

User U0001:

- Modern Fabric Chair
- Vintage Wood Sofa
- Glass Wardrobe
- Modern Table

User U0002:

- Fabric Chair
- Wooden Table
- Leather Sofa
- Classic Wardrobe

These outputs demonstrate that recommendations are influenced by both user similarity and interaction strength.

6. Conclusion

The Week 3 system introduces major improvements over previous iterations:

- Hybrid recommendation improves personalization and ranking quality
- Feature engineering enhances the quality of input signals
- API deployment enables real-world usability

The system now represents a complete pipeline:

Data Processing → Feature Engineering → Modeling → Evaluation → Deployment

7. Future Improvements

- Incorporate content-based filtering using product attributes
- Implement deep learning models such as Neural Collaborative Filtering
- Introduce time-aware recommendations using temporal data
- Deploy system on cloud infrastructure for scalability
- Optimize performance for large-scale datasets